Priority Queues and Heaps

Prof. Timothy Sun

Priority Queues

A (min-oriented) ***priority queue*** is a collection of object and “priority” pairs that supports the following operations:

- insert(*key*, *priority*)
- removeMin( ) - take out element with smallest priority

Priority Queues

A (min-oriented) ***priority queue*** is a collection of object and “priority” pairs that supports the following operations:

- insert(*key*, *priority*)
- removeMin( ) - take out element with smallest priority

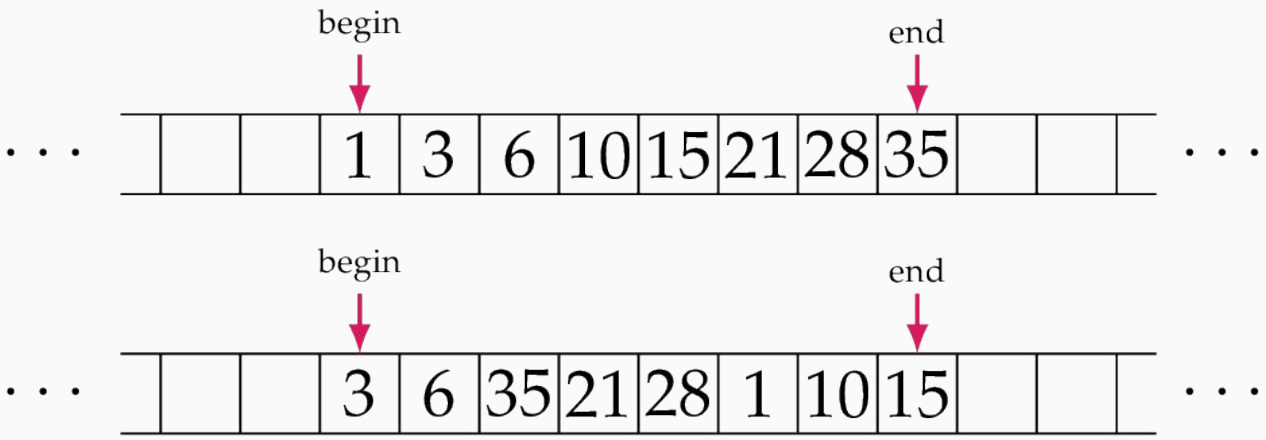
How does one ***implement*** a priority queue?

Array/List-based implementations

Like stacks, queues, etc., a natural attempt would be to use arrays or lists:

Array/List-based implementations

Like stacks, queues, etc., a natural attempt would be to use arrays or lists:



Sorted and unsorted implementations both seem plausible.

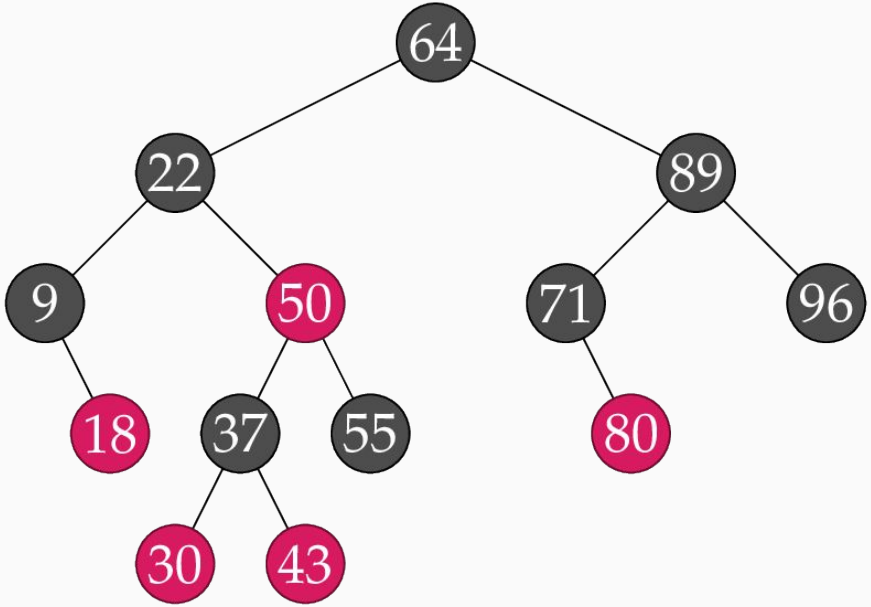
Array/List-based implementations

Each implementation has its strength, and a huge drawback:

	Unsorted List	Sorted List
insert	O(1)	**O(*n*)**
removeMin	**O(*n*)**	O(1)

Balanced BSTs?

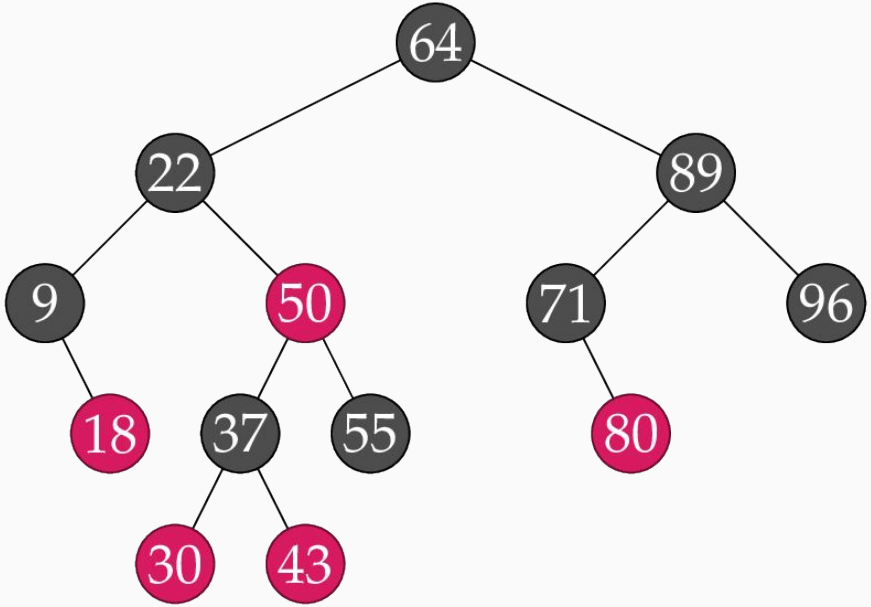
One data structure that would solve this issue is a ***balanced binary search tree*** (e.g. AVL, red-black, etc.)



Balanced BSTs?

One data structure that would solve this issue is a ***balanced binary search tree*** (e.g. AVL, red-black, etc.)

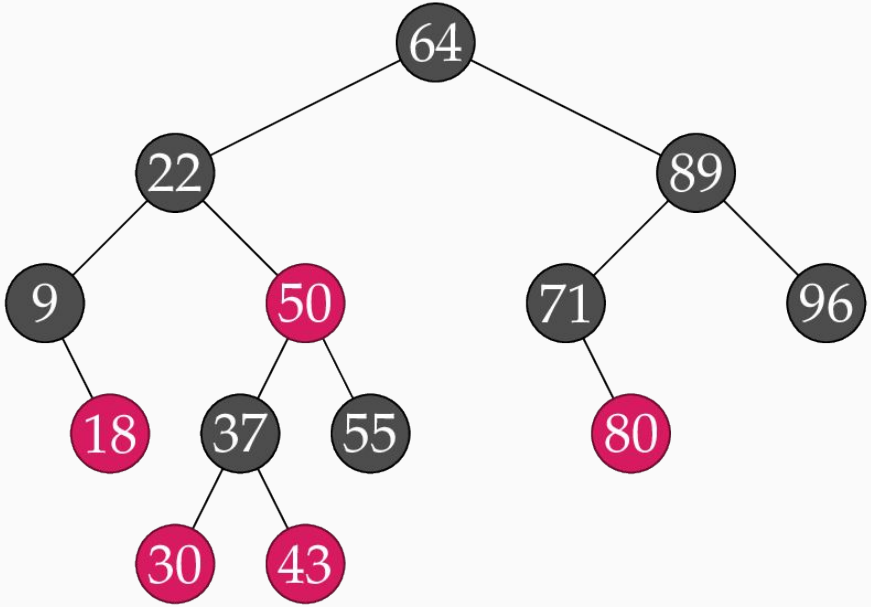
insert and removeMin can be implemented in O(***h***) time.



Balanced BSTs?

One data structure that would solve this issue is a ***balanced binary search tree*** (e.g. AVL, red-black, etc.)

insert and removeMin can be implemented in O(***h***) = O(log ***n***) time.

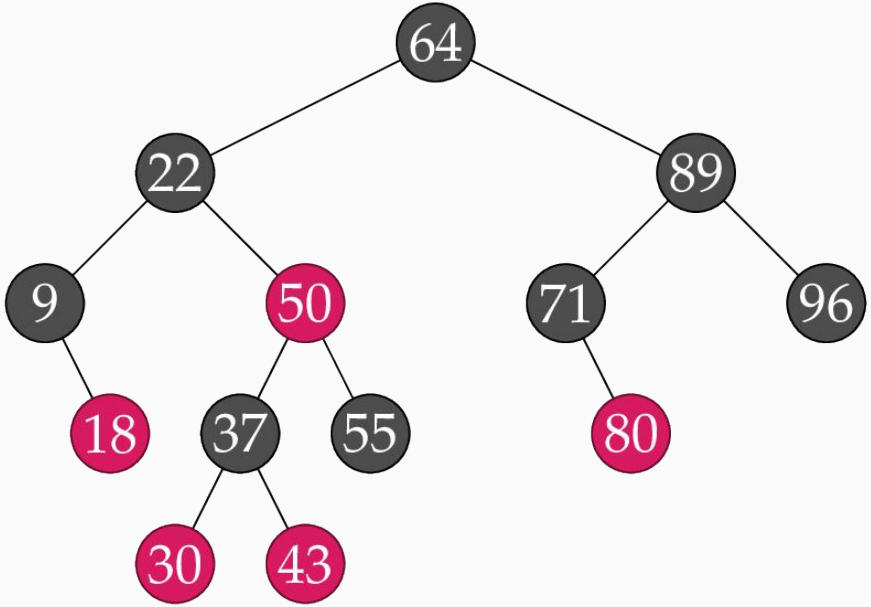


Balanced BSTs?

One data structure that would solve this issue is a ***balanced binary search tree*** (e.g. AVL, red-black, etc.)

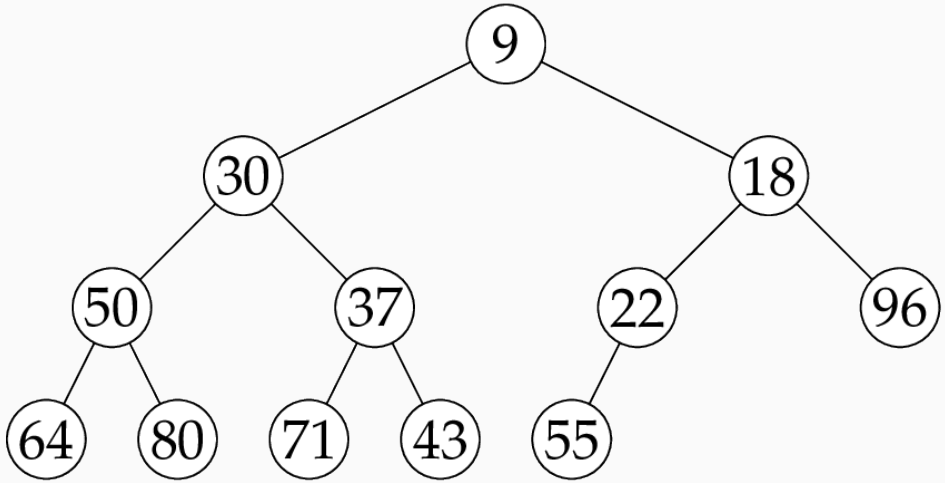
insert and removeMin can be implemented in O(***h***) = O(log ***n***) time.

But this is a little overkill…



Heaps

A ***(binary) heap*** is another tree-based data structure for priority queues.

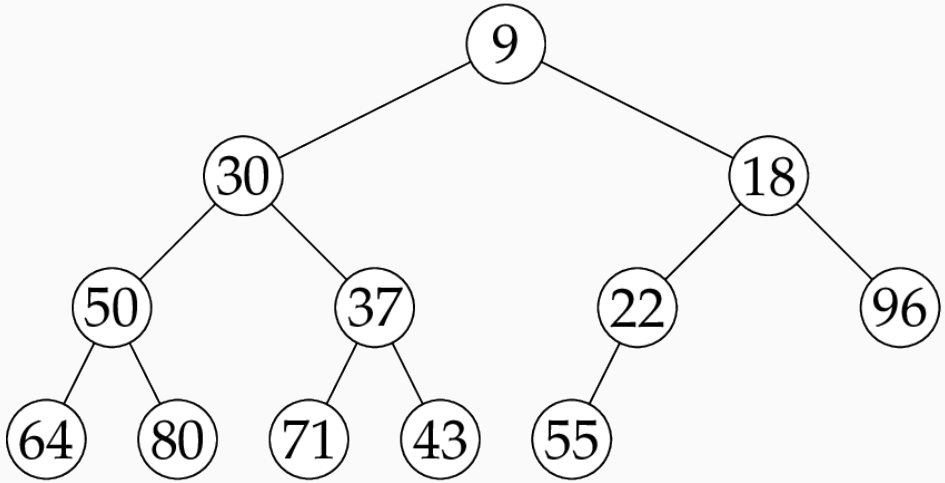


Heaps

A ***(binary) heap*** is another tree-based data structure for priority queues.

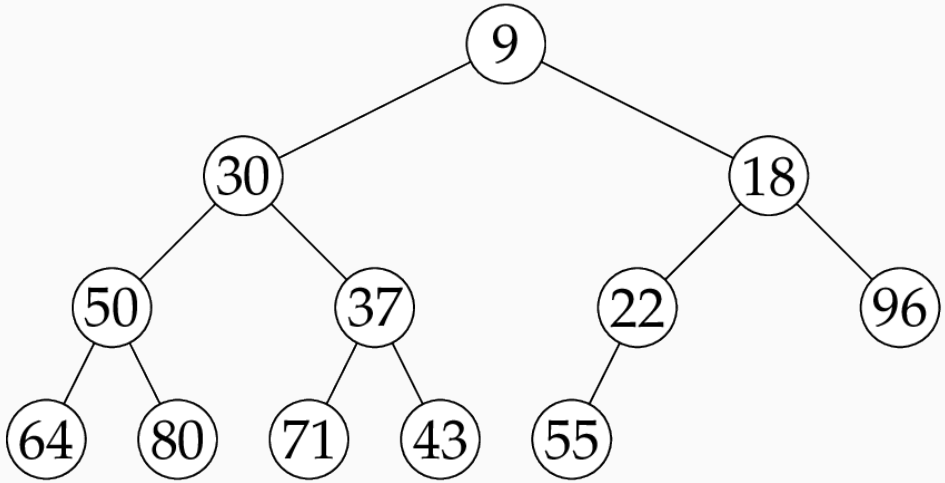
Two key properties:

- Heap order
- Complete binary tree



Heap order property

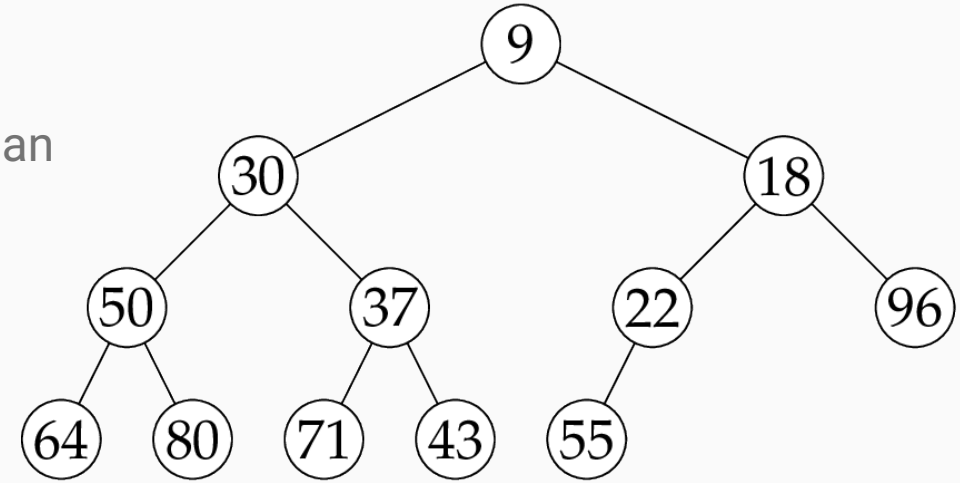
Heap order property: parent always smaller than child



Heap order property

Heap order property: parent always smaller than child

- Grandparent always smaller than grandchild
- … and so on …

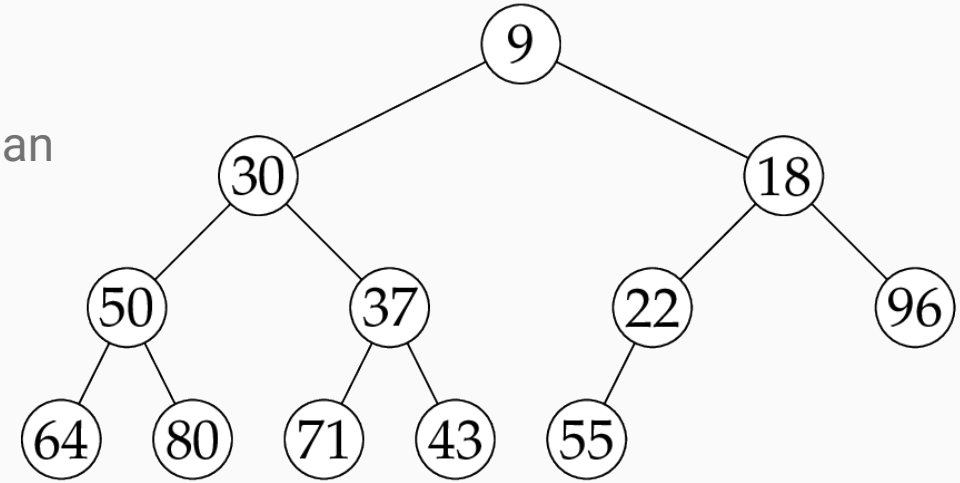


Heap order property

Heap order property: parent always smaller than child

- Grandparent always smaller than grandchild
- … and so on …

Where's the smallest element?



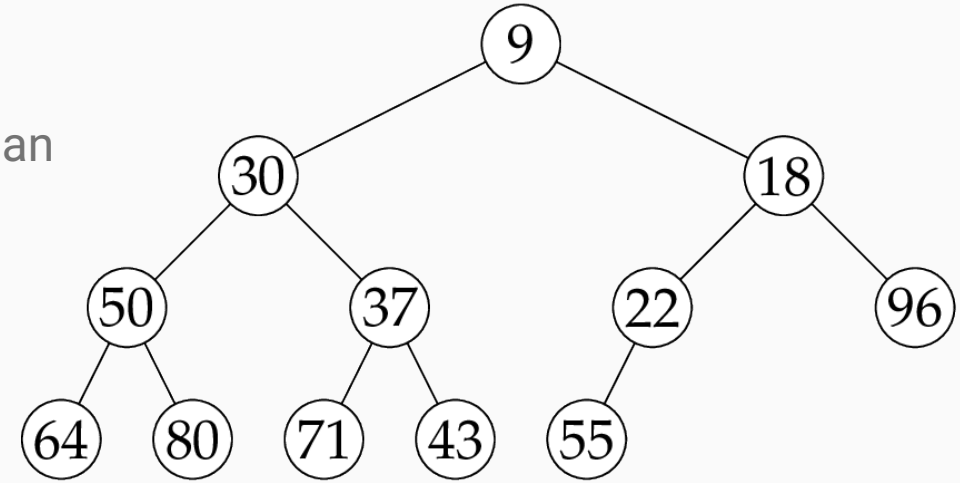
Heap order property

Heap order property: parent always smaller than child

- Grandparent always smaller than grandchild
- … and so on …

Where's the smallest element?

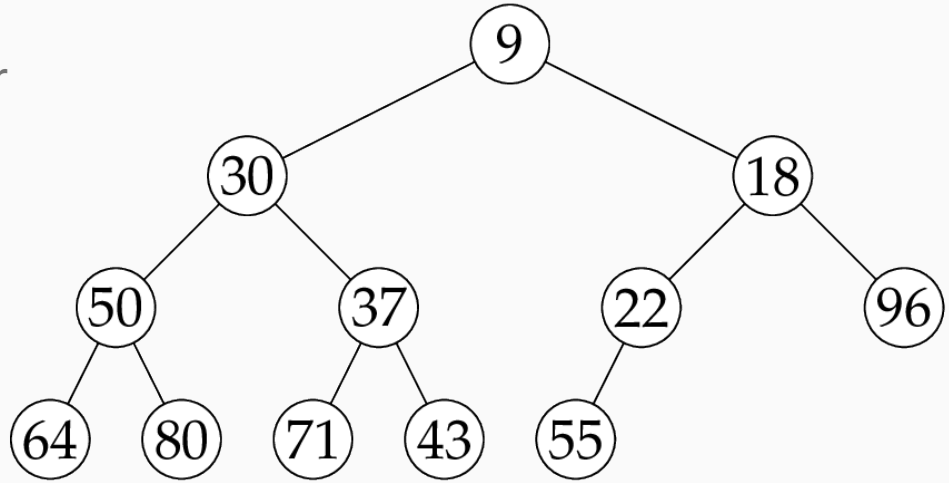
At the root.



Complete binary tree

Complete binary tree property:

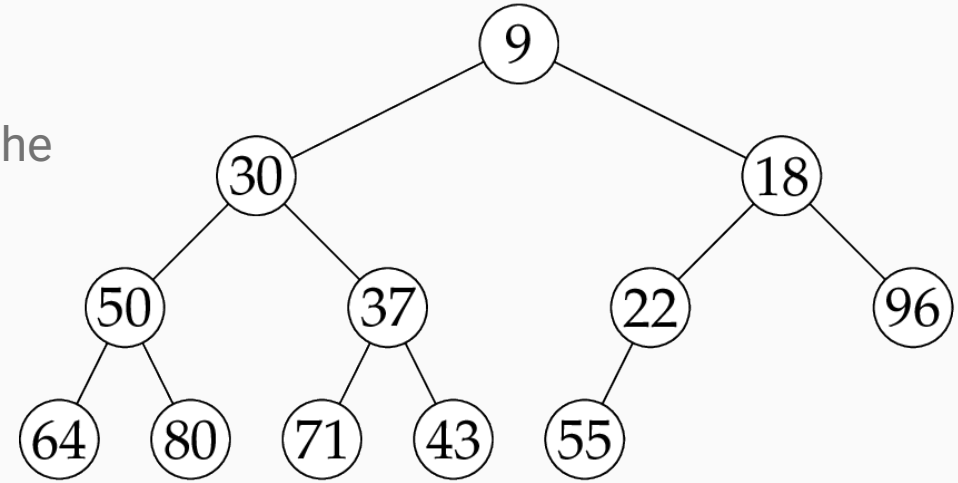
- Except for last layer, every layer is “full”
- All the leaves appear as far left as possible



insert( ) - bubbling up

Each operation must preserve the two properties.

How to add a node that preserves the ***complete binary tree property***?

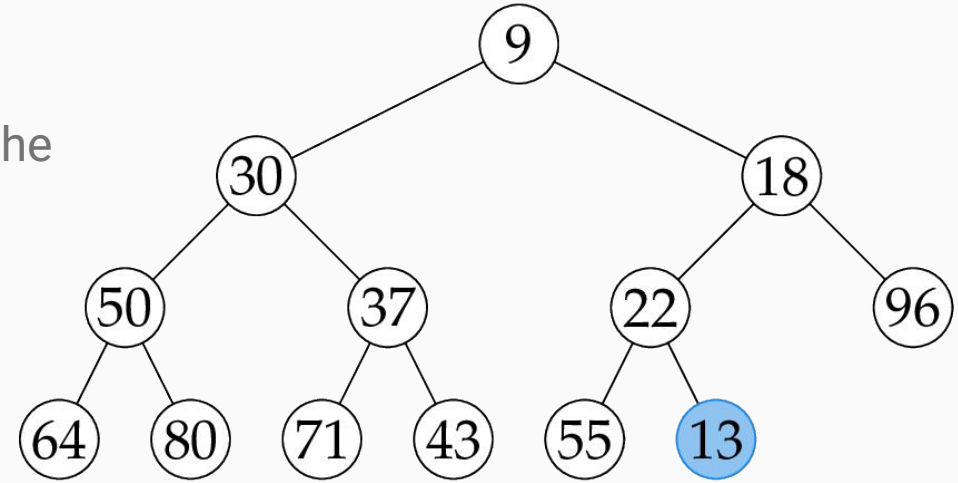


insert( ) - bubbling up

Each operation must preserve the two properties.

How to add a node that preserves the ***complete binary tree property***?

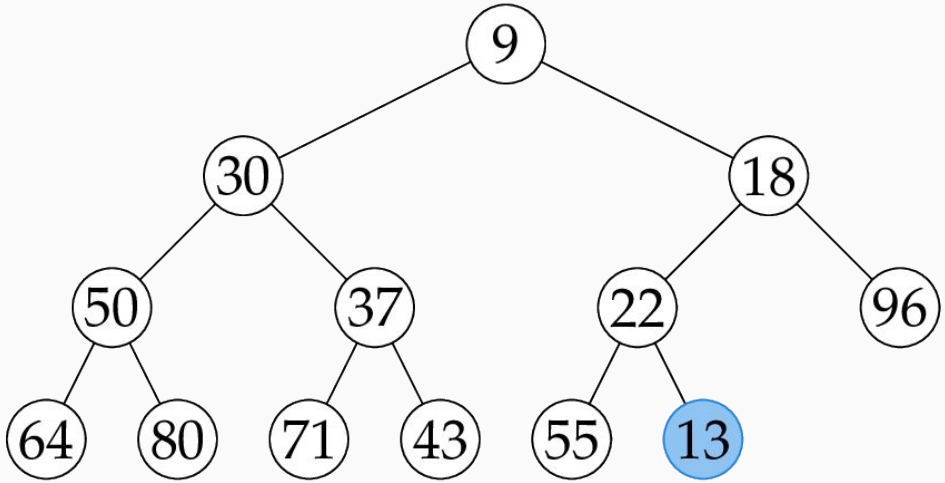
Add to bottom row.



insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

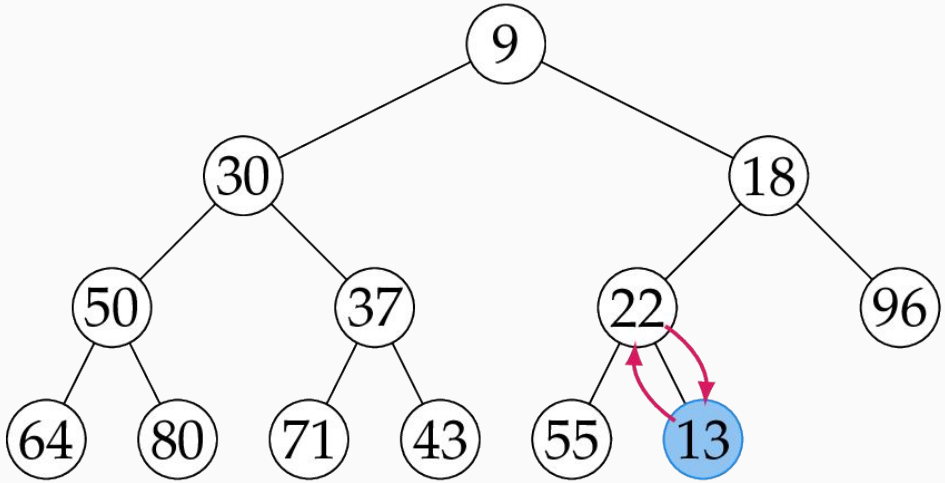


insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

Swap with parent.

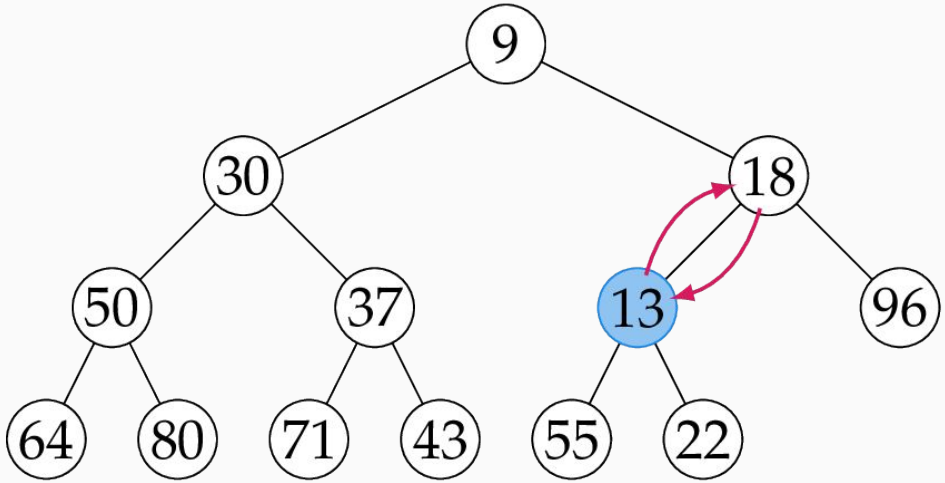


insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

Swap with parent... as many times as necessary

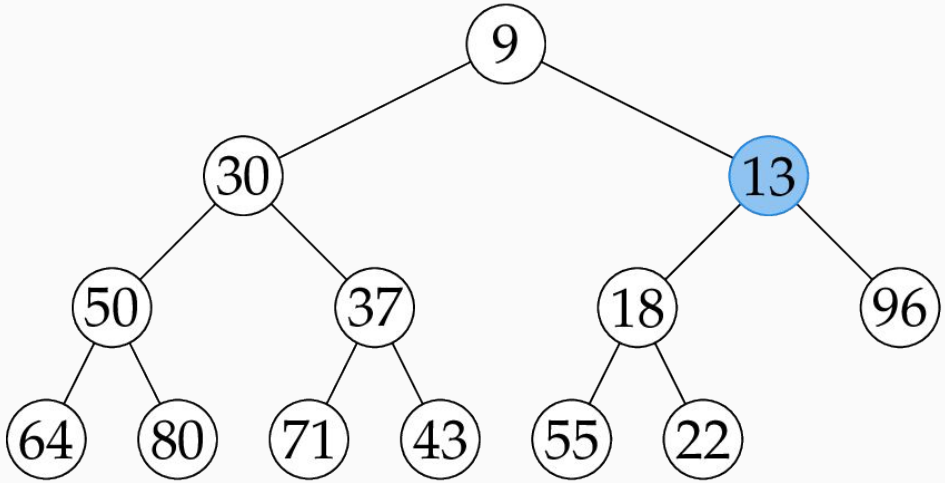


insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

Swap with parent... as many times as necessary

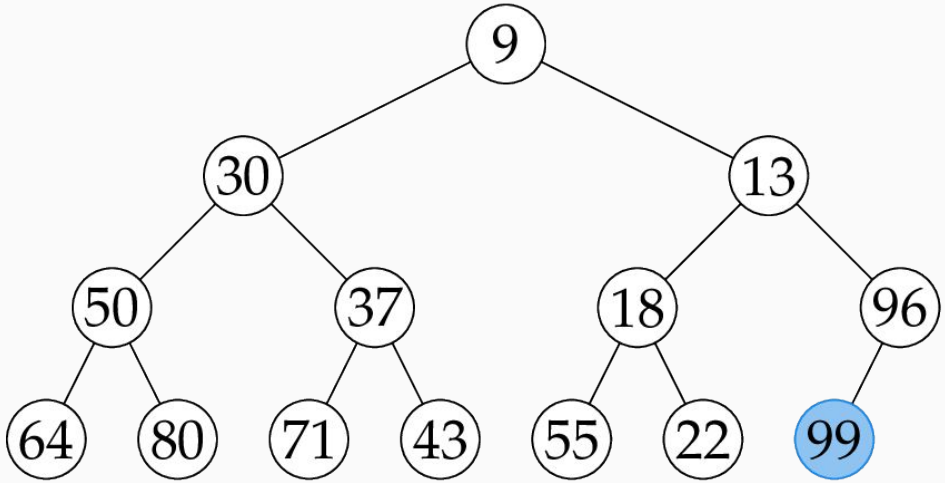


insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

Swap with parent... as many times as necessary

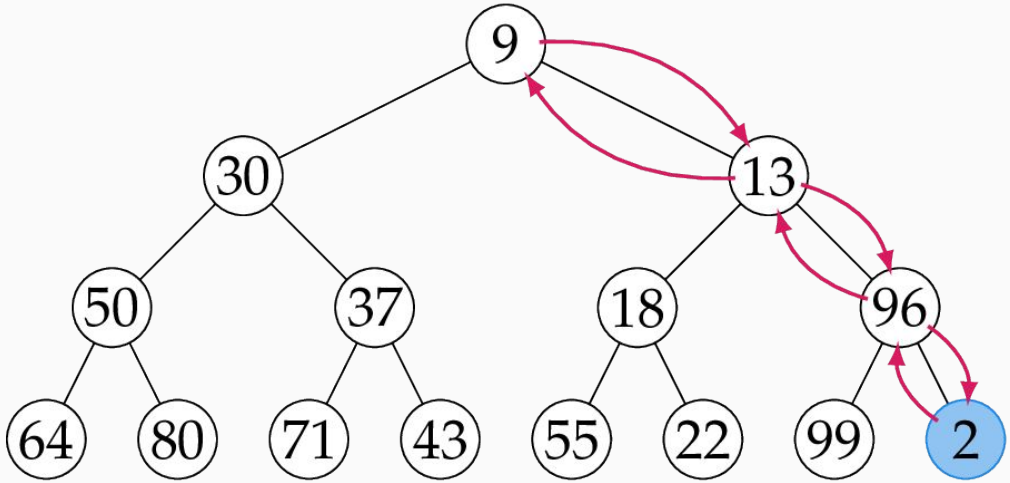


insert( ) - bubbling up

Each operation must preserve the two properties.

How to re-establish the ***heap order property***?

Swap with parent... as many times as necessary

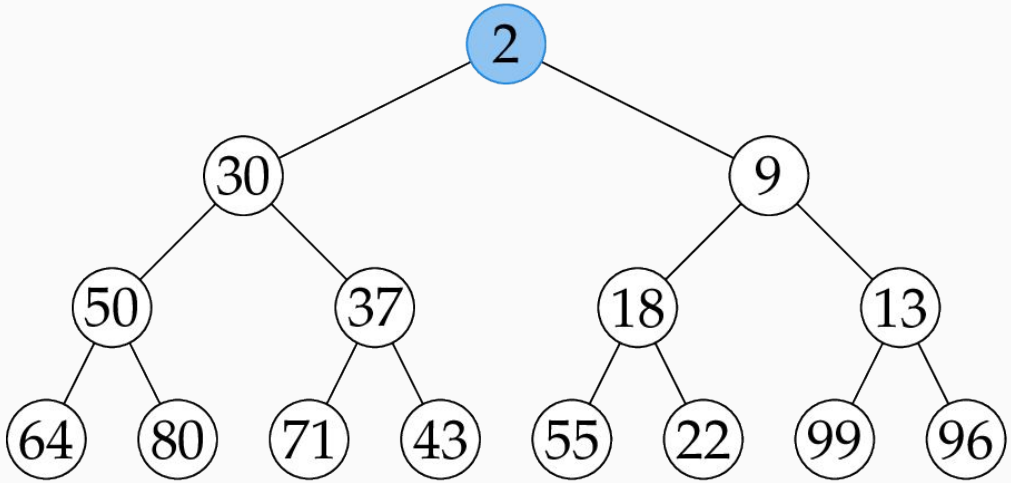


insert( ) - bubbling up

Each operation must preserve the two properties.

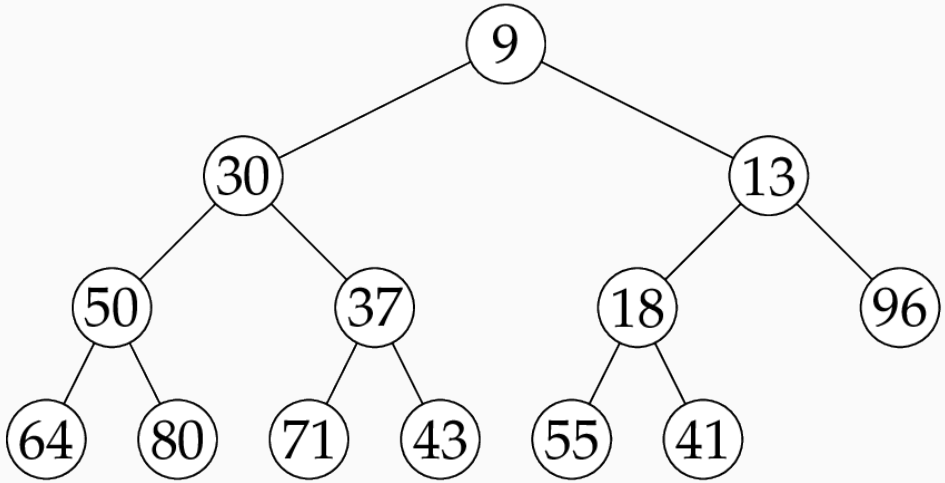
How to re-establish the ***heap order property***?

Swap with parent... as many times as necessary



removeMin( )

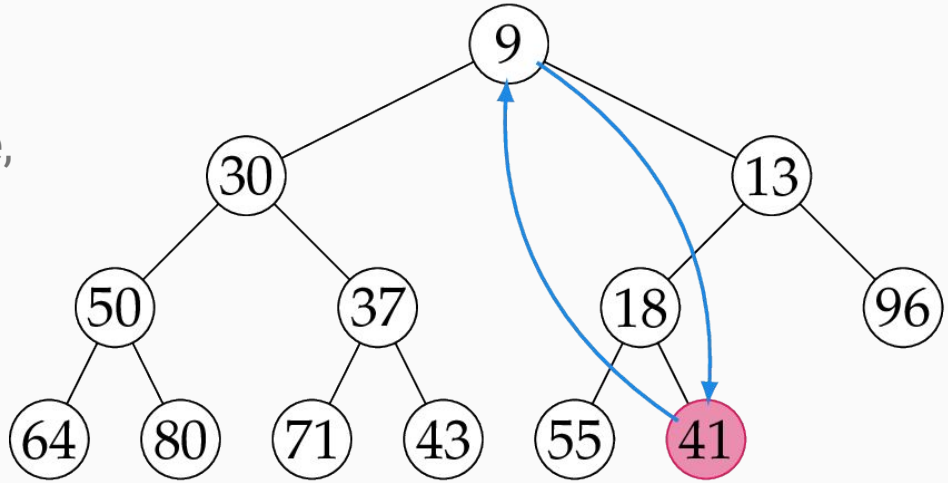
Once again, need to preserve both properties.



removeMin( )

Once again, need to preserve both properties.

Only removable node is the last one, so let's swap it with the root and delete the root.

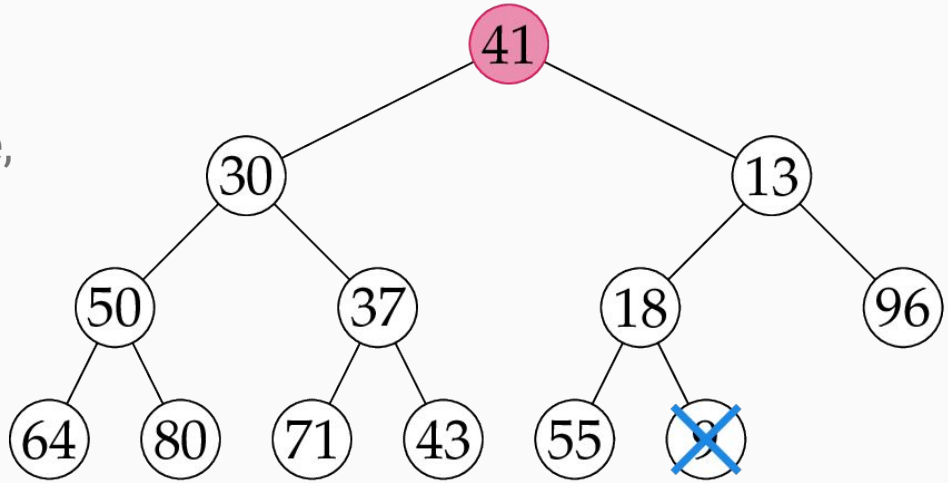


removeMin( )

Once again, need to preserve both properties.

Only removable node is the last one, so let's swap it with the root and delete the root.

Repeatedly swap with smaller child

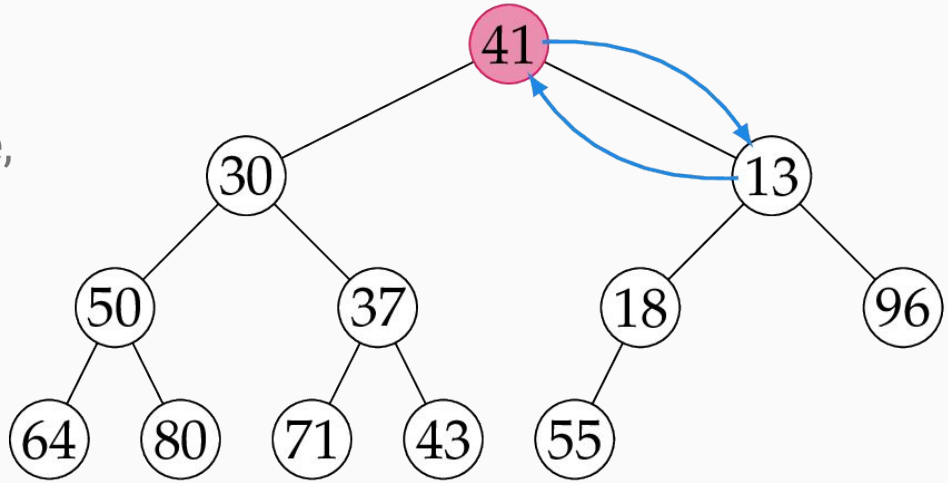


removeMin( )

Once again, need to preserve both properties.

Only removable node is the last one, so let's swap it with the root and delete the root.

Repeatedly swap with smaller child

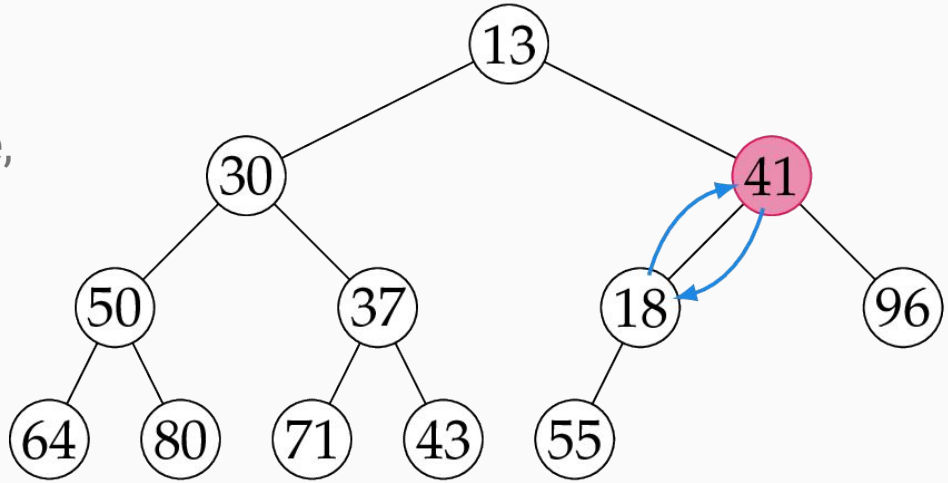


removeMin( )

Once again, need to preserve both properties.

Only removable node is the last one, so let's swap it with the root and delete the root.

Repeatedly swap with smaller child

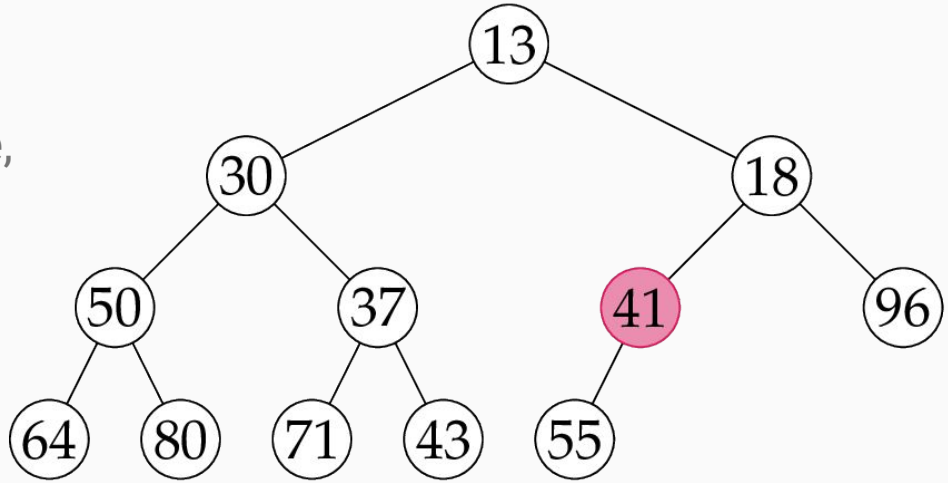


removeMin( )

Once again, need to preserve both properties.

Only removable node is the last one, so let's swap it with the root and delete the root.

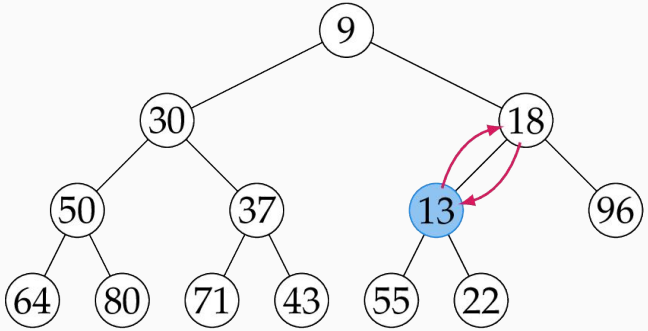
Repeatedly swap with smaller child

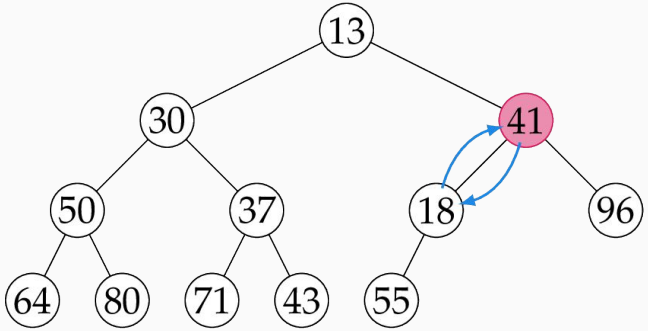


Time complexity

Nodes can “bubble” up or down the entire height of the tree in the worst case.

Both operations take O(***h***) = O(log ***n***).



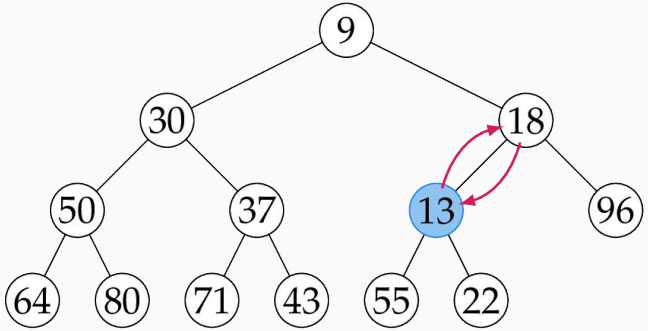


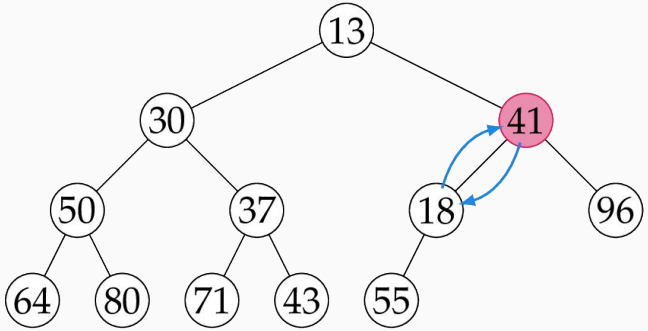
Time complexity

Nodes can “bubble” up or down the entire height of the tree in the worst case.

Both operations take O(***h***) = O(log ***n***).

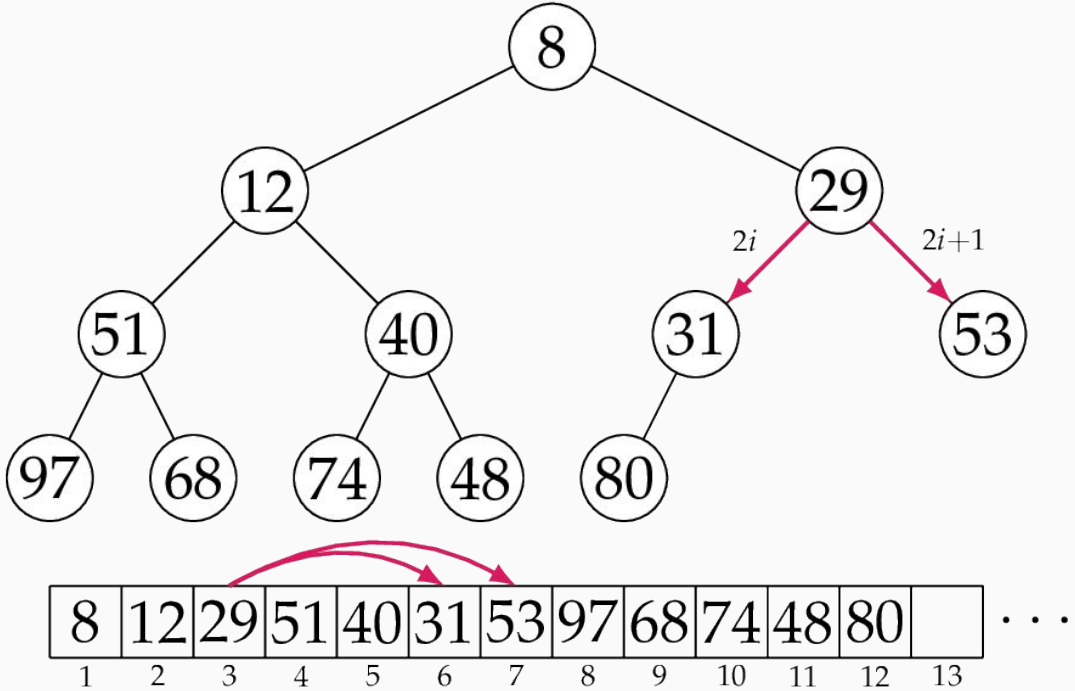
(better hidden constants than BST)





Array implementation

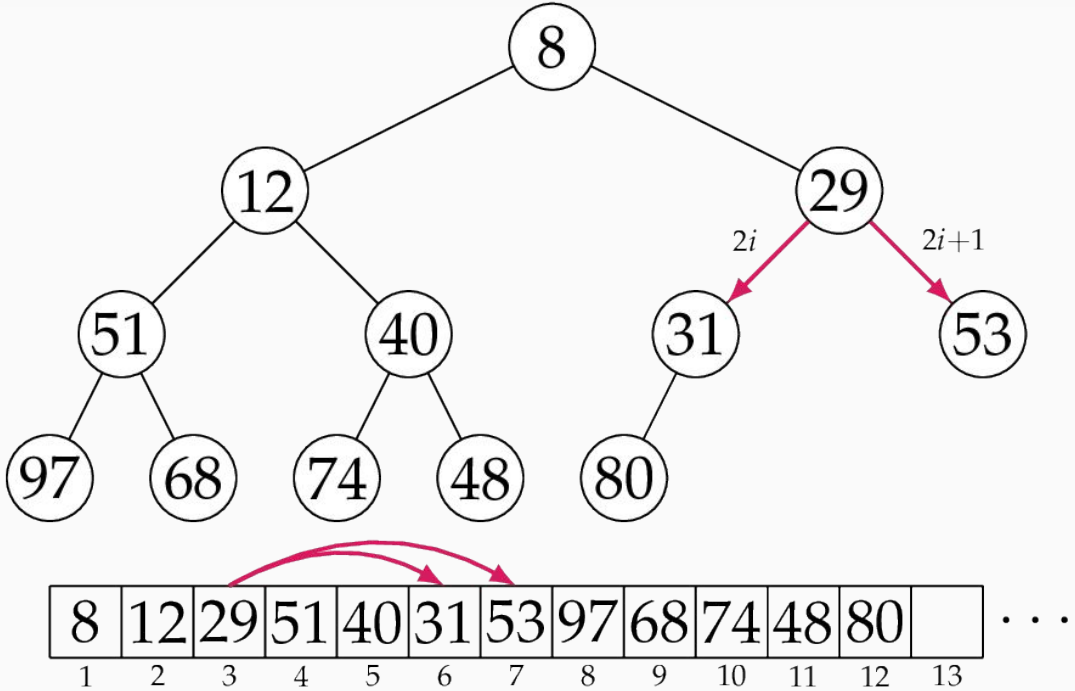
The **complete binary tree** property allows for an array-based implementation.



Array implementation

The **complete binary tree** property allows for an array-based implementation.

No need for node objects, pointers/references, etc.

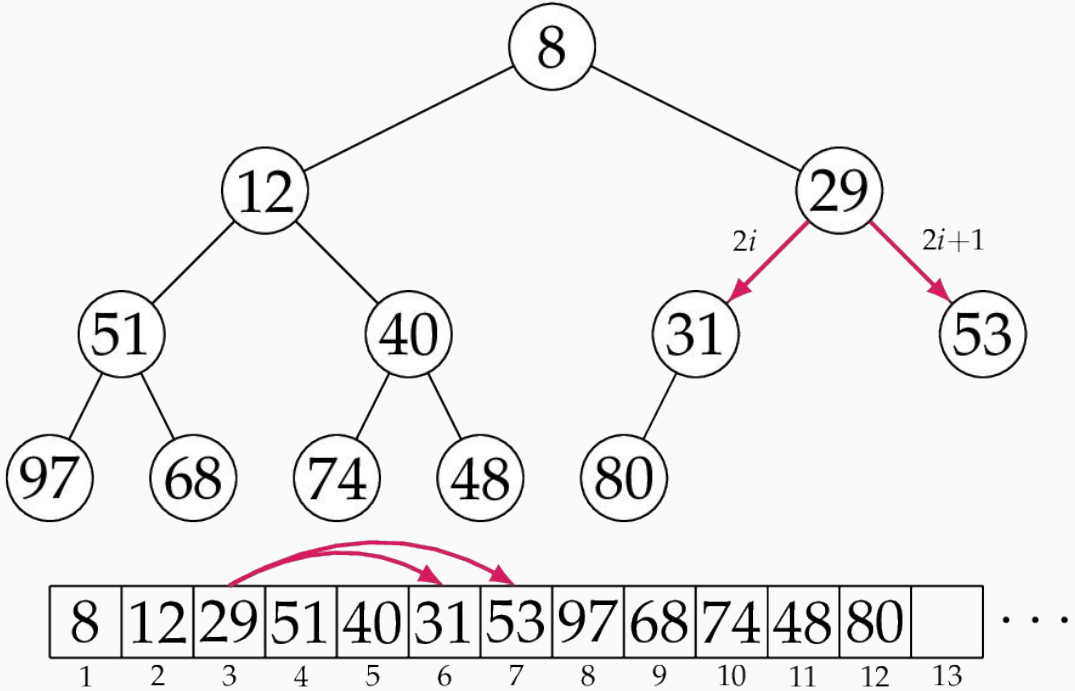


Array implementation

The **complete binary tree** property allows for an array-based implementation.

No need for node objects, pointers/references, etc.

Why isn't this good for all binary trees?



Implementations

Heaps don't have any super-slow operations:

	Unsorted List	Sorted List	Heap
insert	O(1)	**O(*n*)**	O(log ***n***)
removeMin	**O(*n*)**	O(1)	O(log ***n***)

Implementations

Heaps don't have any super-slow operations:

	Unsorted List	Sorted List	Heap
insert	O(1)	**O(*n*)**	O(log ***n***)
removeMin	**O(*n*)**	O(1)	O(log ***n***)

… but these other approaches aren't entirely useless

Heapify

Building a heap from unsorted array in ***linear*** time:

- From bottom layer to top, bubble down

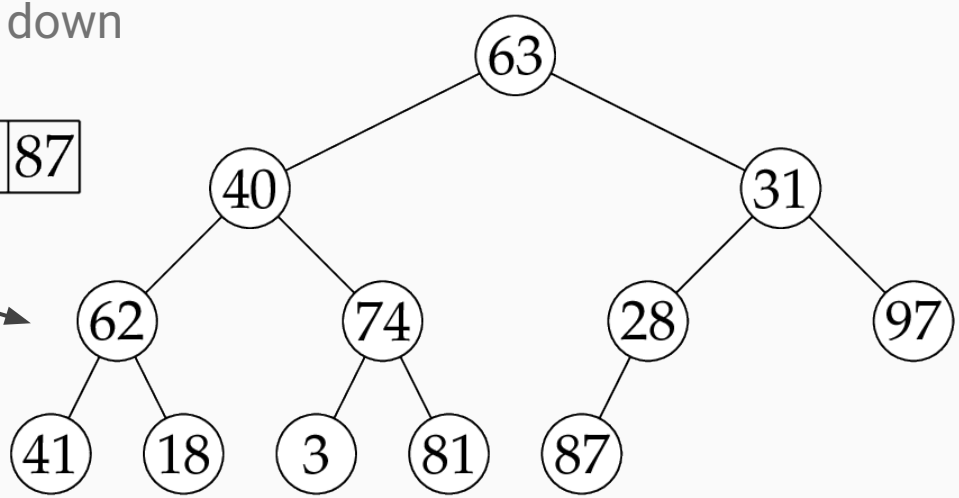
63	40	31	62	74	28	97	41	18	3	81	87

Heapify

Building a heap from unsorted array in ***linear*** time:

- From bottom layer to top, bubble down

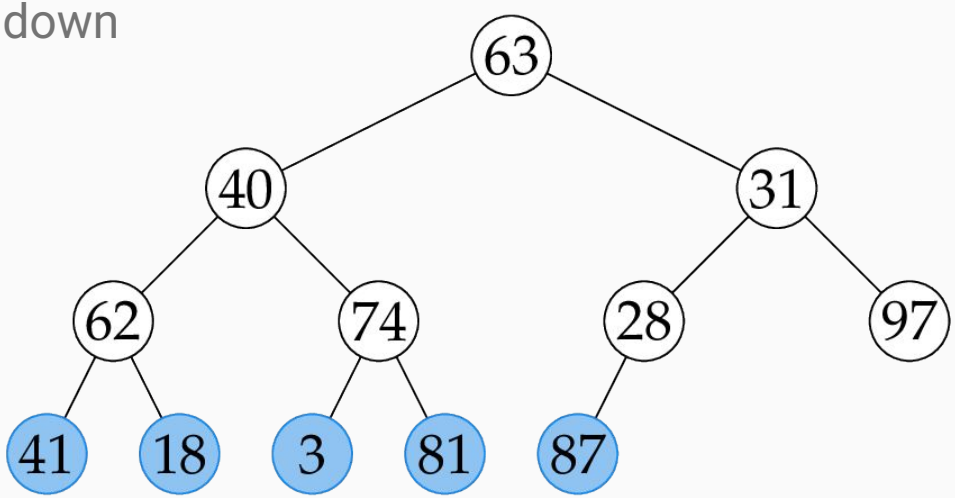
63	40	31	62	74	28	97	41	18	3	81	87



Heapify

Building a heap from unsorted array in ***linear*** time:

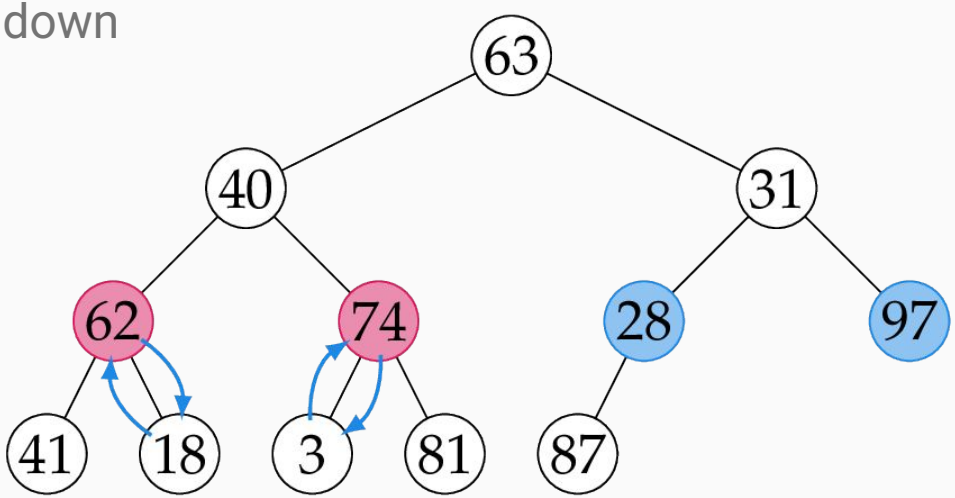
- From bottom layer to top, bubble down



Heapify

Building a heap from unsorted array in ***linear*** time:

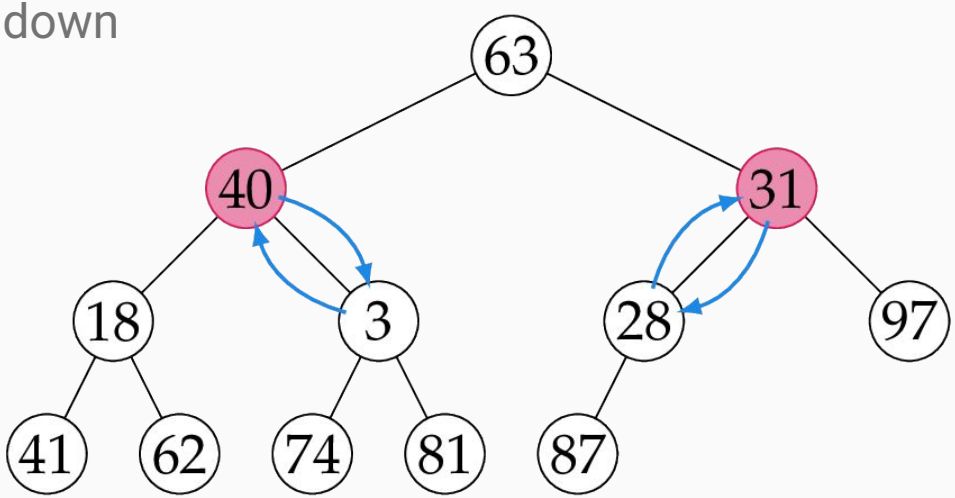
- From bottom layer to top, bubble down



Heapify

Building a heap from unsorted array in ***linear*** time:

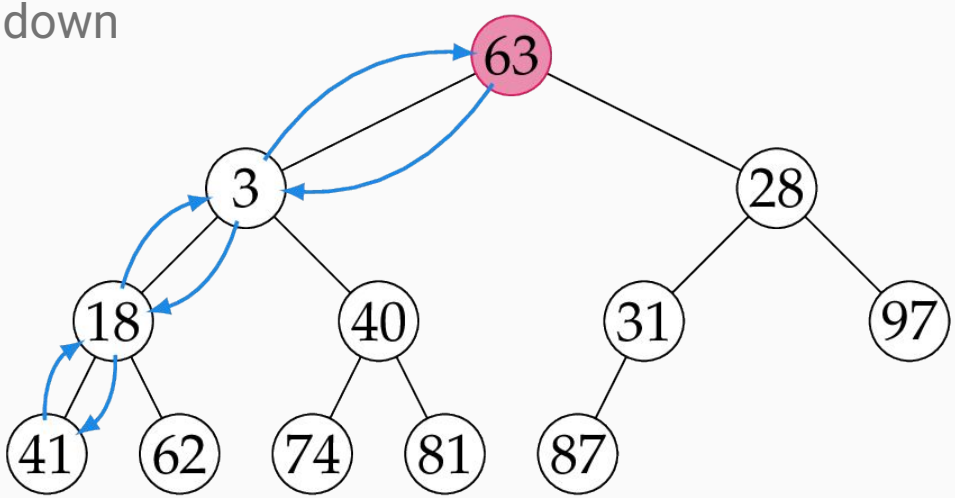
- From bottom layer to top, bubble down



Heapify

Building a heap from unsorted array in ***linear*** time:

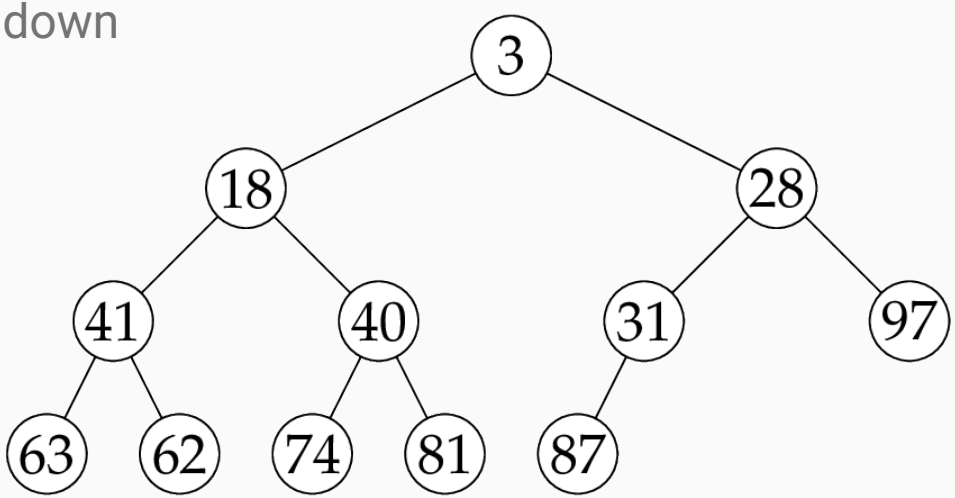
- From bottom layer to top, bubble down



Heapify

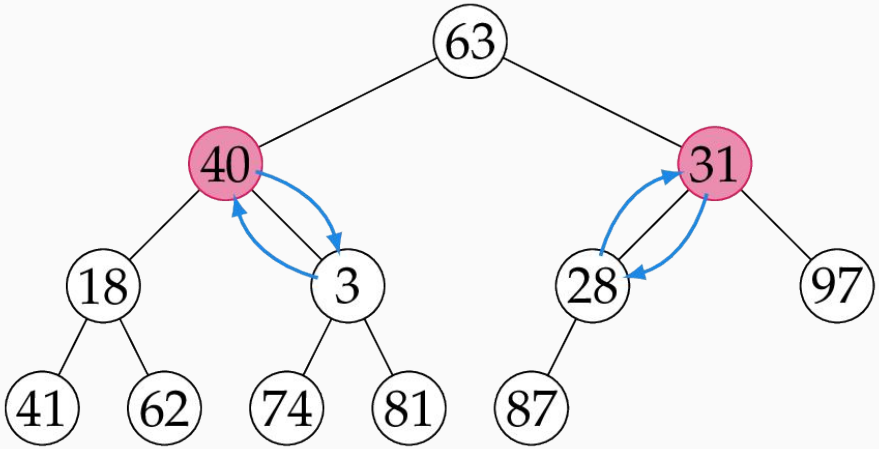
Building a heap from unsorted array in ***linear*** time:

- From bottom layer to top, bubble down



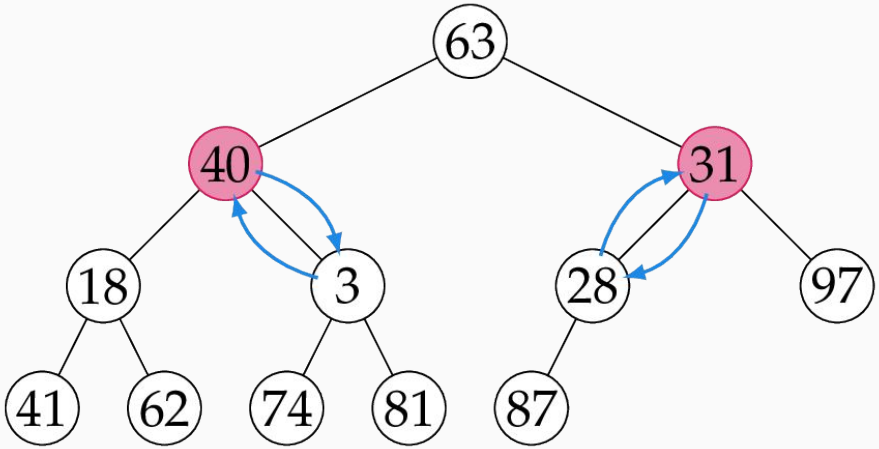
Heapify - correctness

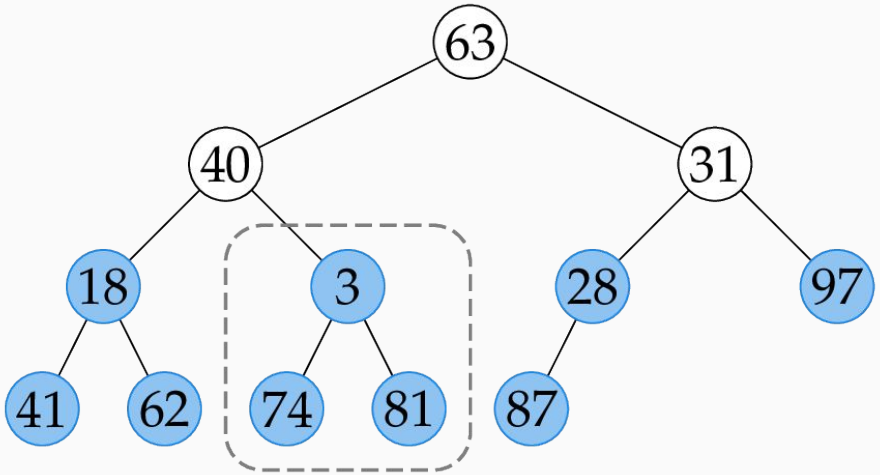
Why does it work? Several “mini-heaps” are created at each layer.



Heapify - correctness

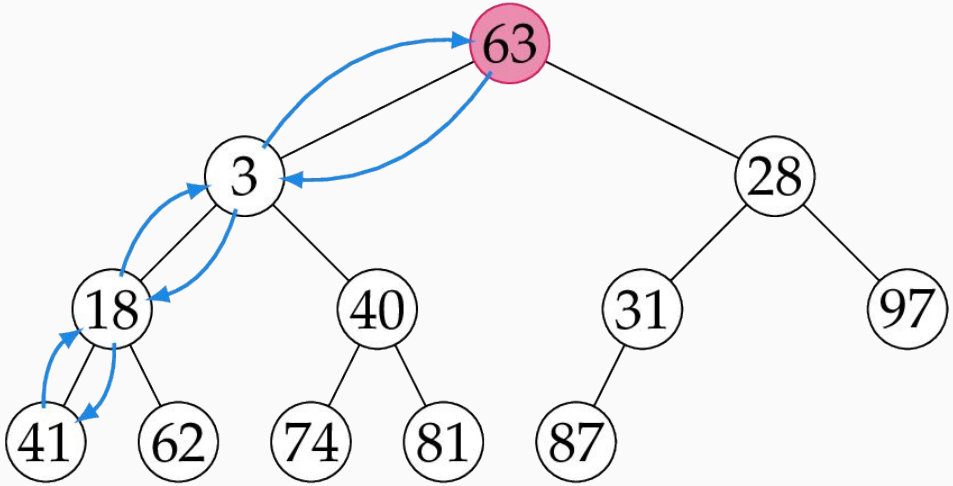
Why does it work? Several “mini-heaps” are created at each layer.





Heapify - runtime

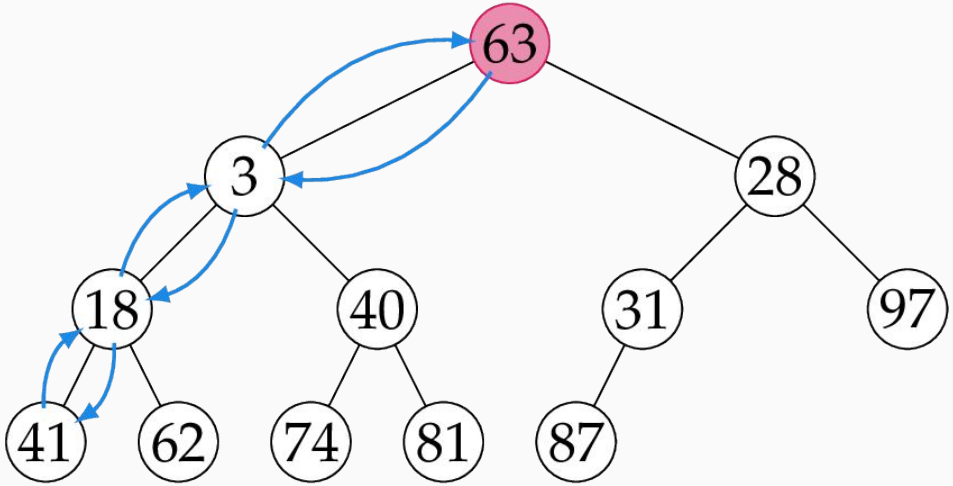
What’s the runtime?



Heapify - runtime

What's the runtime?

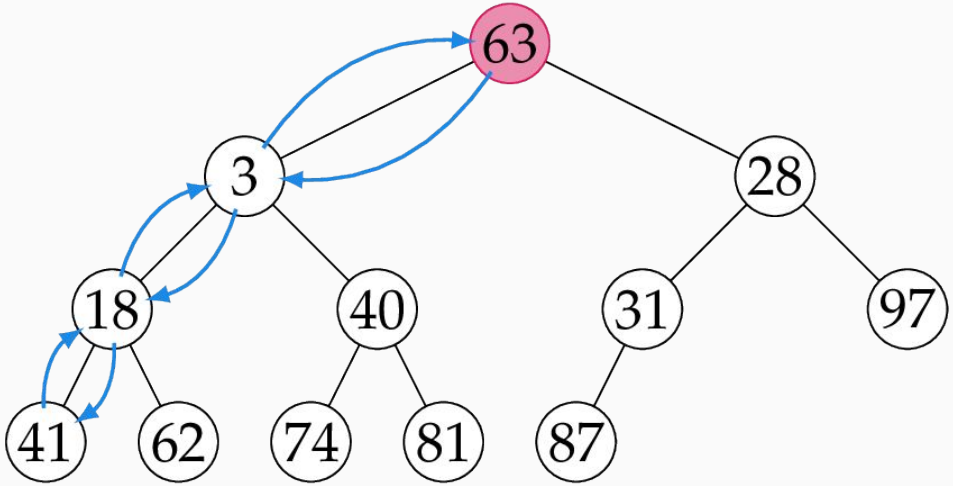
- ***n*** bubbling operations (O(log ***n***)?)



Heapify - runtime

What's the runtime?

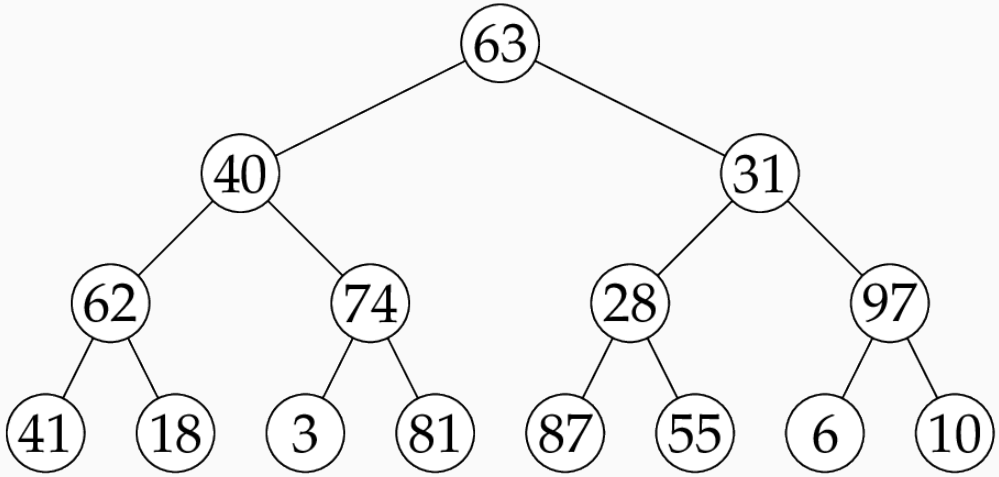
- ***n*** bubbling operations (O(log ***n***)?)
- O(***n*** log ***n***) time???



Heapify - runtime

Suppose bottom layer is full and consider, for each node, the following path:

1. Go to right child
2. Repeatedly go to the left child until you reach a leaf

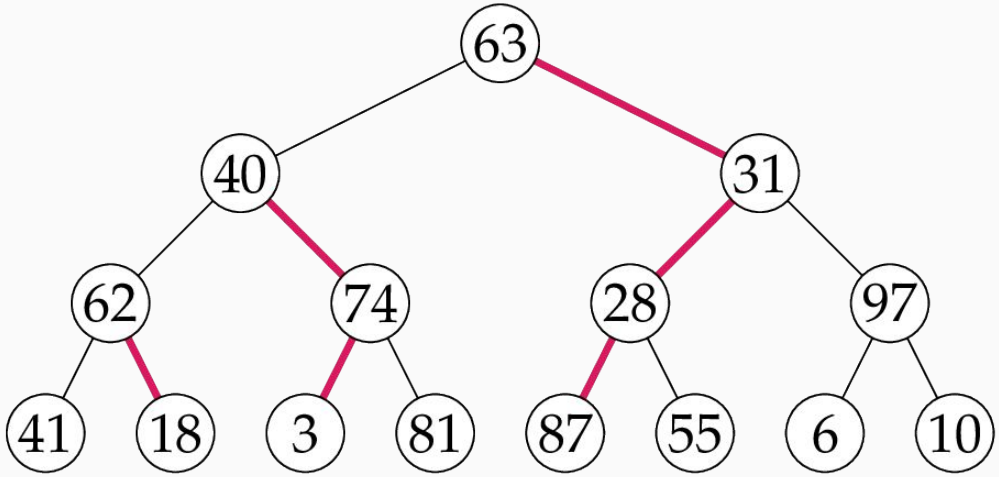


Heapify - runtime

Suppose bottom layer is full and consider, for each node, the following path:

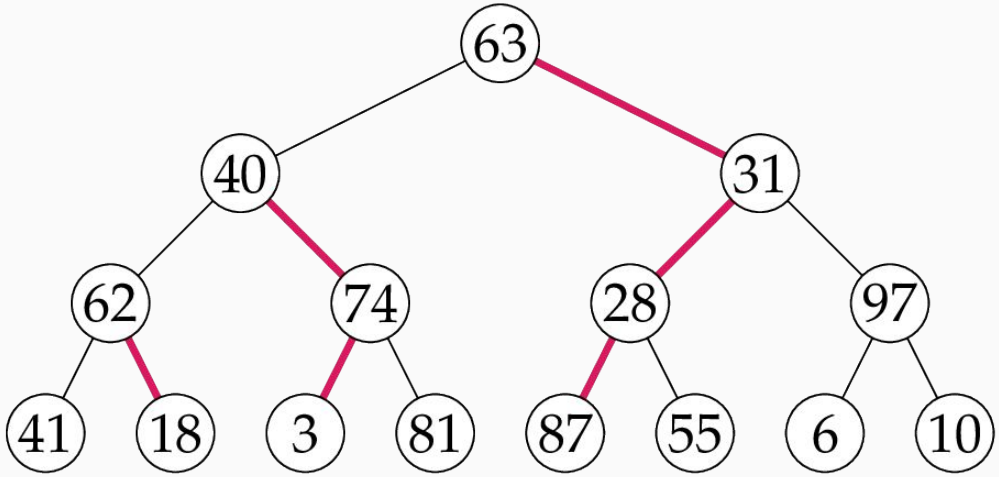
1. Go to right child
2. Repeatedly go to the left child until you reach a leaf

Each path is a potential “bubble-down”



Heapify - runtime

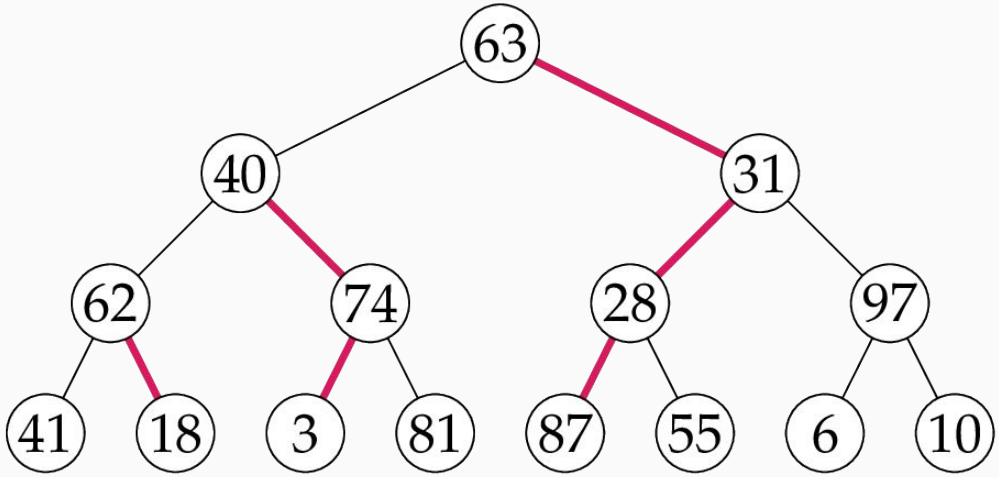
None of these paths overlap, so each edge is used at most once.



Heapify - runtime

None of these paths overlap, so each edge is used at most once.

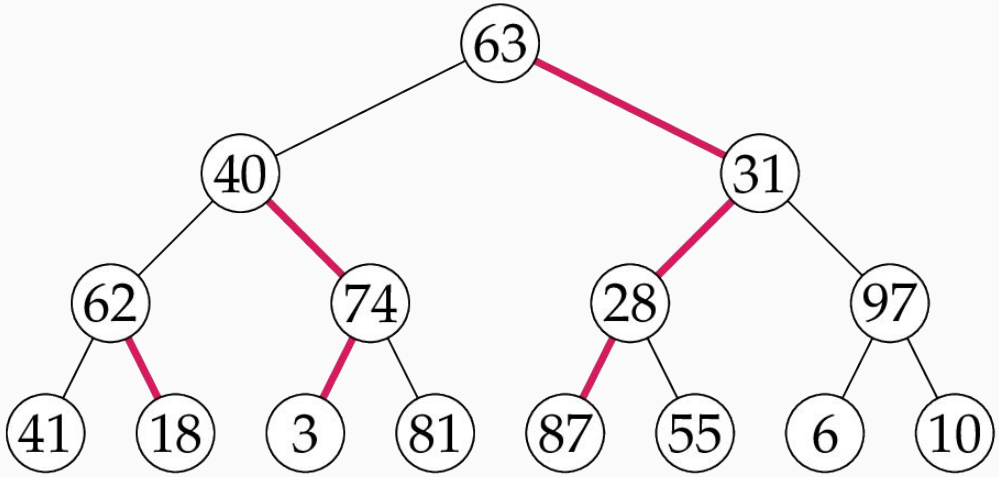
$|E| = n - 1$



Heapify - runtime

None of these paths overlap, so each edge is used at most once.

$$|E| = n - 1 \rightarrow O(n)$$



Heapsort

A simple sorting algorithm using a priority queue:

- Build a priority queue with all the elements inside.
- Repeatedly call removeMin( )

Heapsort

A simple sorting algorithm using a ***heap***:

- Build a ***heap*** with all the elements inside. → O(***n***)
- Repeatedly call removeMin( ) → ***n*** O(log ***n***) = O(***n*** log ***n***).

Total runtime: O(***n*** log ***n***)

“Extended” heap operations

In a binary heap, possible to modify other elements besides the root:

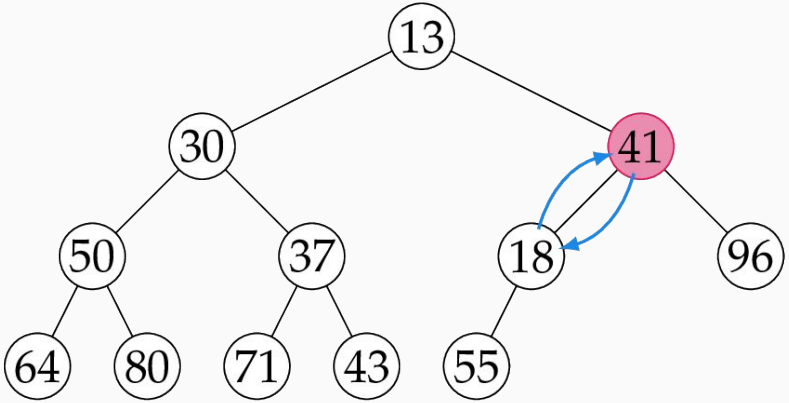
- decreaseKey(*key*, *newPriority*)
- remove(*key*)

“Extended” heap operations

In a binary heap, possible to modify other elements besides the root:

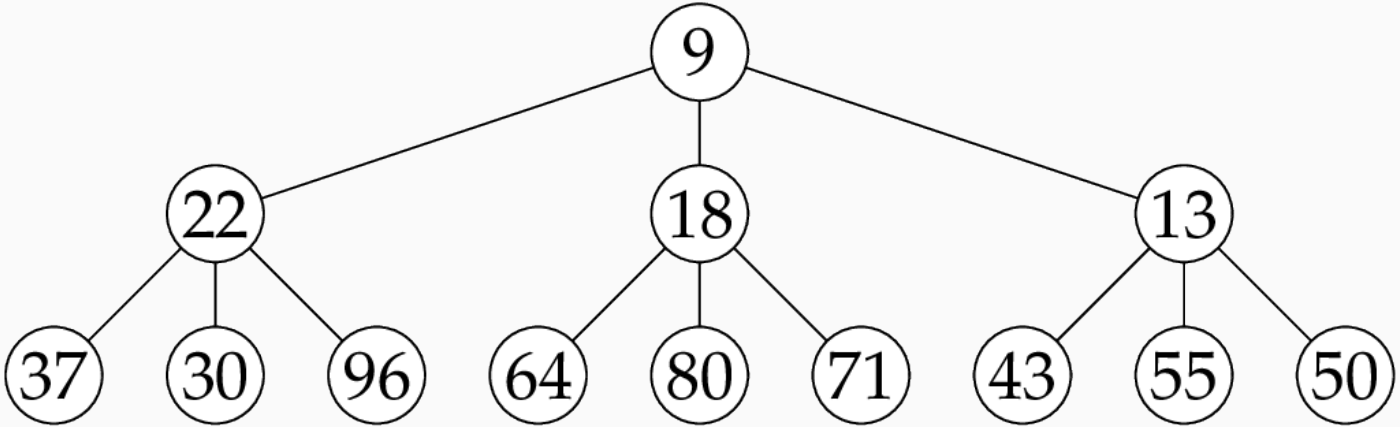
- decreaseKey(*key*, *newPriority*)
- remove(*key*)

Uses same bubbling operations as before



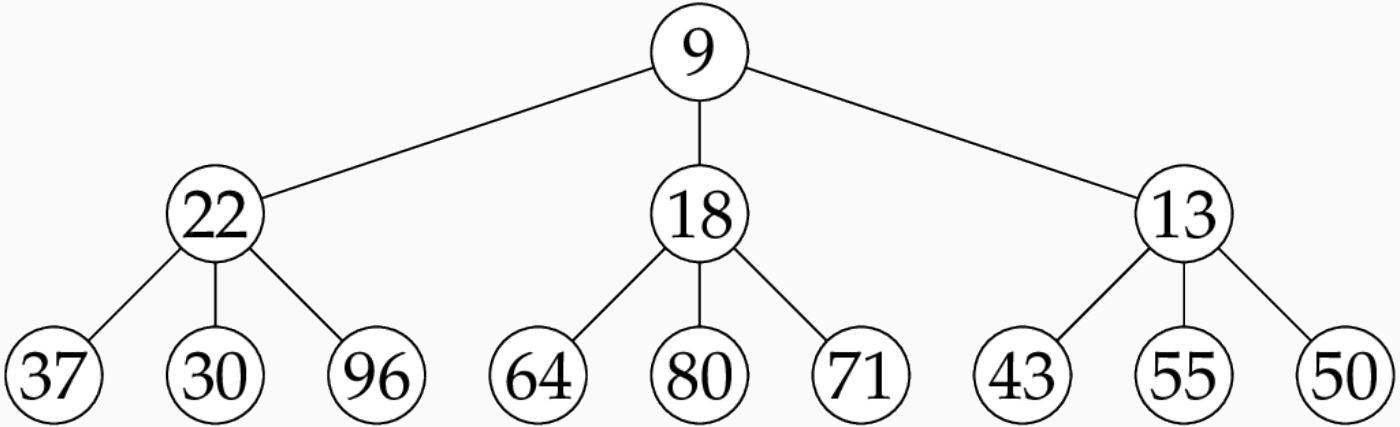
d-ary heaps

No reason why a heap can't have more than two children (what about BSTs?)



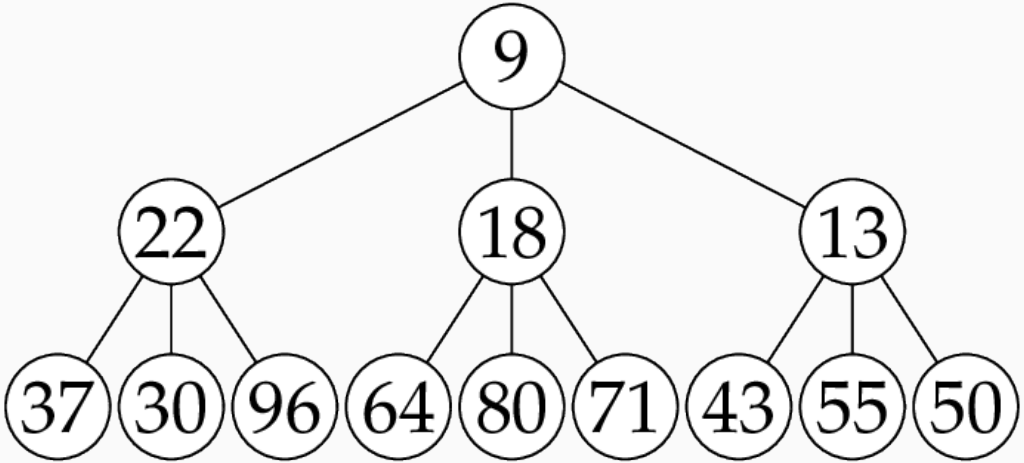
d-ary heaps

No reason why a heap can't have more than two children (what about BSTs?)



How does this affect runtime?

d-ary heaps

Still O(log ***n***), but maybe you care about hidden constants.

- Bubbling up is shorter.
- Bubbling down is longer.